LongConspectWriter: Automatic Generation of Structured Lecture Conspects on Consumer GPUs

Alexander Volzhanin

GitHub · Article on Habr

June 2026

Abstract

Automatic generation of structured academic conspects from audio recordings of lectures in the exact and natural sciences is difficult for local small language models (SLMs). The transcript of a lecture lasting ≈1.5 h amounts to roughly 15–20 thousand tokens and formally fits within the context window of modern local SLMs; however, when processing such a context, a single-call SLM systematically degrades: it loses fragments from the middle of the sequence, fails to retain structure, and hallucinates terms and formulas. This is a manifestation of the Lost in the Middle effect: information-retrieval accuracy follows a U-shaped curve — high at the edges of the context and dropping in the middle; in our setting we use SLMs whose behaviour is likewise described in that work, and which exhibit pronounced forgetting not only from the middle but also from the beginning of the context. Moreover, on an 8 GB VRAM budget a single-call over a long transcript is practically infeasible, which makes decomposition not an optimization but a necessary condition for operability.

This work proposes the LongConspectWriter project (hereafter LCW) — a multi-agent pipeline in which no single LLM call ever receives the full transcript, and the structure of the conspect is formed before synthesis begins. The key mechanism is two-level planning on top of two-level semantic clustering: the transcript is segmented into chronologically contiguous local clusters, a short description is generated for each, a global chapter plan is built from the set of descriptions, and the local clusters are then distributed across the chapters. Synthesis is performed chunk-by-chunk in a MapReduce scheme with a registry of named entities that prevents re-introducing concepts.

To assess quality, a set of seven conspecting paradigms is formulated and used as a system of criteria. In a preliminary evaluation under the LLM-as-a-judge methodology, LCW scores on average 6.87/10 against 8.84/10 for Gemini 3.1 Pro (the SOTA model from Google at the time of this work), which processed the same transcript in a single call — corresponding to ≈78% of the reference quality at zero inference cost and fully locally. Notably, the characteristics least affected were exactly those most vulnerable to long-context degradation: informational completeness and self-sufficiency showed the closest match to the reference (with the caveat that high completeness is partly built into the method itself — see Section 7), whereas the main shortfall fell on factual accuracy and mathematical rigour, which are limited rather by the size of the local model than by the pipeline architecture. Visualization is low for both systems — automatically choosing an appropriate chart type and integrating it correctly into the conspect is an objectively hard task regardless of model size.

The test sample comprised only 10 lectures, so no statistical conclusions can be drawn: the figure of 78% should be treated as a preliminary result for forming a hypothesis. The source code, the testing dataset, and the tests are available under the MIT license on GitHub.

Contents

1. Introduction

The nature of long-context degradation has been studied by Liu et al.: on multi-document QA and key-value retrieval tasks, information-retrieval accuracy follows a U-shaped curve — high at the edges of the context and dropping in the middle of the window. In the original work this is demonstrated on MPT-30B-Instruct, LongChat-13B, GPT-3.5-Turbo, and Claude-1.3. In our setting a small (8B) quantized model is used. On top of the degradation itself, the 8 GB VRAM budget adds a resource constraint: a single-call over a long transcript on the target hardware is additionally infeasible in terms of memory and speed (detailed analysis below). This feasibility consideration is secondary to the degradation itself and only reinforces the conclusion: here decomposition is not an optimization but a necessary condition.

Feasibility of a single-call on the target hardware. Direct processing of the full transcript (˜15–20 thousand tokens) by the target model T-lite (8B) on an RTX 3050 8 GB testbed is unrealizable for two distinct reasons, depending on the execution format. In the non-quantized format (transformers, fp16) the weights of the 8B model alone exceed 8 GB VRAM, and the run fails with an out-of-memory error (OOM) before generation even begins. In the quantized format (GGUF Q5_K_M via llama-cpp-python 0.3.20) the model fits in memory, but the working pipeline configuration uses a context window of 8192 tokens, into which the full tran-

script (15–20 thousand tokens) does not fit; a single-call would require a many-fold larger `n_ctx`, whose KV cache on top of the weights either does not fit in 8 GB or forces some layers to be offloaded to RAM — whereupon the generation speed drops to a few tokens per second. That is, in GGUF a single-call is not so much impossible as practically unacceptable in terms of time. Switching to a substantially smaller model to reduce memory requirements yields an unusable output. The practical consequence: on this budget a single-call is not a viable alternative, and therefore the correct quality reference is a cloud model (Section 6) rather than a single-call of the same model.

Hence the central idea of this work — to build the structure of the conspect before synthesizing the text. The intuition is simple: if the model cannot reliably hold the entire transcript at once, then it should not see it in its entirety — first a "map" of the lecture is compiled from short fragments, and only then, following this map, is the text written fragment by fragment.

The simplest form of such decomposition — naive map-reduce, the independent compression of chunks followed by concatenation — is feasible on the target hardware but unsatisfactory for a lecture: each call sees only its own short fragment and has no notion of the lecture as a whole, so the result falls apart into disconnected pieces with repeated definitions and without any control over coverage completeness. The bottleneck here is not the context length of an individual call (each chunk is already short), but the inconsistency between calls: none of them knows either its place in the lecture or what has already been said in the others. LCW eliminates precisely this: first a chapter plan — a map of the whole lecture — is assembled from short cluster descriptions, and only then, chapter by chapter, the text is synthesized under this plan. The plan is exactly the shared knowledge that naive map-reduce lacks; meanwhile the full transcript still never enters any single call.

This work considers the task under a setting that simultaneously fixes three hard constraints: fully local execution without cloud APIs or paid subscriptions; 8 GB VRAM — the most common amount of video memory among Steam users (≈27%, Steam Hardware Survey, April 2026); robustness to long context for lectures lasting 1.5 h.

The contribution of this work is as follows:

(1) LongConspectWriter is proposed — a fully local multi-agent pipeline in which no single LLM call receives the full transcript, and the structure of the conspect is formed before text synthesis begins;

(2) a mechanism of two-level hierarchical planning on top of chronologically constrained semantic clustering is proposed, allowing a global conspect plan to be built without feeding in the full transcript;

(3) a set of seven conspecting paradigms is formulated as a system of requirements and criteria for evaluating the quality of academic conspects;

(4) an experimental evaluation under the LLM-as-a-judge methodology is carried out on 10 lectures from five subject domains, compared against a cloud SOTA reference.

2. Related Work

Existing approaches to the automatic conspecting of audio lectures fall into two groups, each of which leaves at least one of the three constraints fixed in the introduction unsolved.

Local pipelines without structure planning. There exist fully local solutions for audio transcription and summarization that do not depend on cloud APIs and that solve the locality

constraint. However, such pipelines build a conspect by directly summarizing the transcript without first constructing a structure, and on a long transcript they inherit the same long-context degradation as single-call feeding: robustness to long context remains unsolved.

Cloud services for generating learning materials. Cloud solutions rely on large models with a big context window and achieve high quality, but violate the requirements of locality and zero inference cost: processing requires sending the recording to a third-party service and a paid subscription.

Positioning of the present work. LCW occupies an intermediate position: while preserving the full locality of the first group, it abandons direct summarization of the long transcript and builds the structure of the conspect before text synthesis, thereby addressing precisely the long-context degradation that the local solutions of the first group leave unattended. In other words, the structure here is not a matter of formatting but a protective mechanism: it fixes what should be said and where, before the SLM begins writing the text and gets the chance to lose something. The novelty of the work lies neither in chunk-wise processing as such (a basic property of hierarchical summarizers) nor in clustering on its own, but in a concrete combination: the global conspect plan is built from short cluster descriptions before text synthesis; local clustering is chronologically constrained by a connectivity matrix; inter-chunk coherence is maintained by a registry of named entities without feeding in the full context; and the entire pipeline is designed for the 8 GB VRAM constraint.

The proposed approach adjoins the line of work on long-context decomposition, in which three directions can be distinguished. Hierarchical and recursive summarization compresses text bottom-up: Wu et al. recursively summarize books, first compressing small fragments and then their summaries, while RAPTOR recursively clusters fragments and builds a tree of summaries. Multi-agent schemes distribute a long input across calls: in Chain of Agents workers read chunks sequentially, passing a compressed message to one another, and a final agent assembles the answer. The plan-before-write line builds structure before text: STORM first forms an article plan and then writes under it from retrieved sources. The local-clustering stage, meanwhile, is akin to classical segmentation of text into subtopics (TextTiling, Hearst), and the two-level "subtopics within topics" structure directly echoes our division into local and global clusters.

Against this background, the novelty of LCW is not algorithmic but systemic-methodological, and it is convenient to decompose it into three levels. First, engineering: none of the listed works targets a hard 8 GB VRAM budget with a fully local SLM (RAPTOR and STORM rely on large cloud models), and the contribution is the very feasibility of such a combination. Second, methodological: unlike RAPTOR's semantic clustering, which merges nearby fragments regardless of their position, LCW uses the temporal monotonicity of a lecture as a structural prior — both clustering (the connectivity matrix) and the assignment of clusters to chapters (chronological smoothing of transitions) are constrained by the flow of time, owing to which the global plan is recovered from short cluster descriptions without feeding in the full transcript. Third, empirical: a set of seven paradigms is proposed as a system of criteria, and a testable hypothesis about the separation of quality axes is formulated (Section 9). The work proposes no algorithmically new components: the contribution lies in tying them together under the three simultaneous constraints fixed in the introduction.

3. Method

3.1. Overview

The pipeline is organized into three sequential blocks: transcription and two-level clustering with planning; text synthesis; and visualization generation with PDF assembly. Each block saves its intermediate result to disk and passes the next block a path to the artifact rather than data in memory; the engineering consequence of this decision for VRAM management is discussed in Section 4.

The transcript of a lecture lasting ≈1.5 h has a size on the order of 15–20 thousand tokens. All text agents of the pipeline (the planners, the synthesizer, the extractor, the visualization planner) use a single model, T-lite-it-2.1, in GGUF Q5_K_M quantization (file `T-lite-it-2.1-Q5_K_M.gguf`) with a context window of 8192 tokens (4096 for the visualization planner) via llama-cpp-python (version 0.3.20); the visualization-code-generation agent is Qwen2.5-Coder-7B-Instruct in Q6_K quantization (context 4096 tokens). Generation by the text agents is greedy (temperature 0, `min_p` 0.15, `repeat_penalty` 1.05; 1.1 for the visualization planner). The overall pipeline scheme is shown in Fig. 1.

As shown in the introduction, both naive approaches are untenable on the target hardware: a single-call is infeasible in memory and speed (Section 1), while bare map-reduce runs but yields a fragmentary conspect without global structure. Therefore LCW builds two mechanisms on top of chunk-wise processing: two-level planning sets the global structure before synthesis, and the entity registry removes duplication. Thus decomposition solves the feasibility problem, while planning and the registry solve the conspect-quality problem.

3.2. Transcription

The audio recording is converted to text by the Whisper `large-v3-turbo` model in the faster-whisper implementation (CTranslate2 format). Before the run, a system prompt is supplied, varied depending on the subject domain of the lecture, which improves the recognition accuracy of domain terminology. The result of this stage is the raw transcript.

3.3. Local semantic clustering

The transcript is segmented into local clusters — chronologically contiguous semantic blocks each covering a single micro-topic. The sentences of the transcript (split using the `razdel` library) are vectorized by the rubert-tiny2 model on the CPU, and clustering is performed by the agglomerative algorithm with a cosine metric, complete linkage, and a distance threshold of 0.5. The number of clusters is not specified in advance but is determined by this threshold.

A key feature is the chronology constraint. Fragments that are not adjacent in time may turn out to be semantically close — for example, when the lecturer returns to a definition — and standard clustering would merge them, violating temporal order. To avoid this, the agglomerative algorithm is run with a connectivity matrix that permits merging only temporally adjacent fragments and forbids the merging of chronologically distant ones. The matrix permits a connection only with the immediate temporal neighbours (two off-diagonals — an adjacency matrix).

```
# Connectivity matrix: each sentence is connected only
```

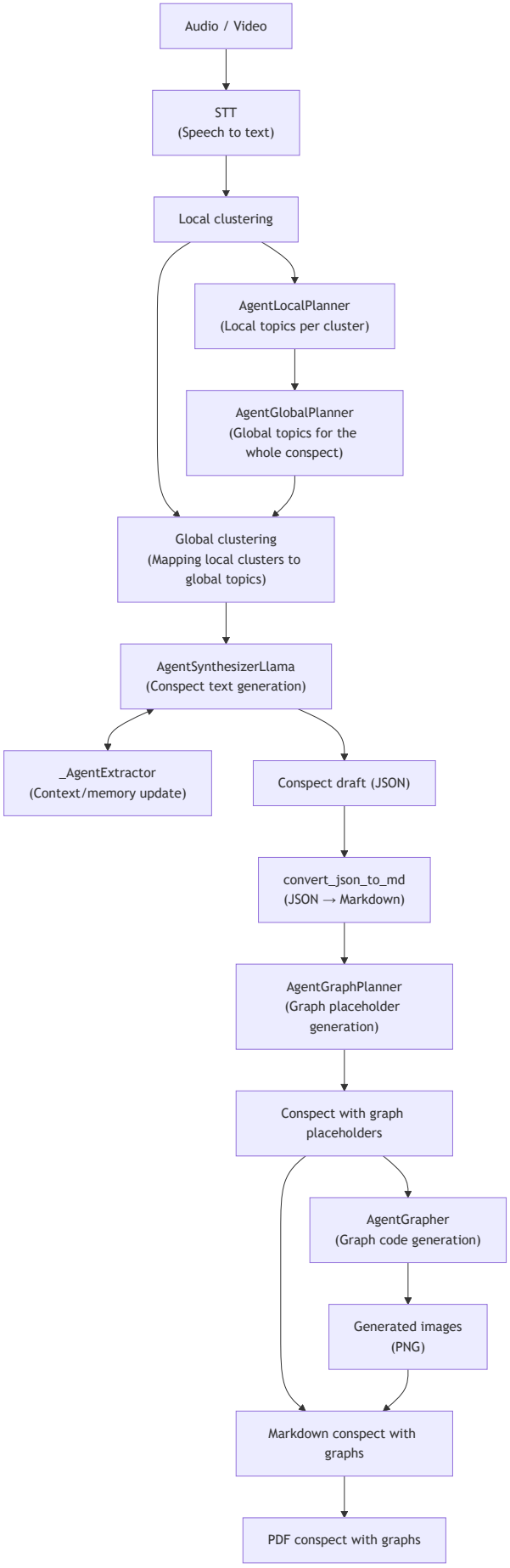

Figure 1. Overall architecture of the LCW multi-agent pipeline.

```
# to its immediate temporal neighbours (i-1 and i+1)
if self._gen_config.turn_on_connectivity:
    connectivity = np.eye(n_samples, k=1) + np.eye(n_samples, k=-1)

local_clusterer = AgglomerativeClustering(
    n_clusters=None,
    distance_threshold=self._gen_config.threshold,  # 0.5
    metric=self._gen_config.metric,                  # cosine
    linkage=self._gen_config.linkage,                # complete
    connectivity=connectivity,
)  # simplified
```

As a result, each local cluster is a contiguous sequence of consecutive sentences.

An illustrative hypothetical example. Suppose that at the start of the lecture the consecutive sentences are “Consider the definition of the limit of a sequence”, “A sequence converges if...”, and “In other words, from some index onward all terms...”. They are semantically close and chronologically adjacent, so they fall into a single local cluster. If, however, forty minutes later the lecturer returns to the topic (“let us recall the definition of a limit, with which we began”), this fragment is semantically close to the first cluster but chronologically distant from it — the connectivity matrix forbids their merging, and the repetition remains in its own, later cluster, preserving the flow of the lecture.

3.4. Two-level hierarchical planning

Local planning. Each local cluster is passed independently to the LocalPlanner agent, which produces a single short string describing what the cluster is about. The agent processes exactly one cluster per call and solves exactly one task. This is consistent with the hypothesis from the introduction: an SLM reliably solves a local task on a short input, whereas a global task on a long input is inaccessible to it without degradation.

A hypothetical example: for the cluster from the previous point, the LocalPlanner might return a string of the form “Definition of the limit of a sequence and its informal interpretation”.

Global planning. The resulting descriptions are collected into a single list, which — and only it, without the original lecture text — is passed to the GlobalPlanner agent. The agent builds from these descriptions a single hierarchical chapter plan for the entire conspect, with precise section headings in JSON format. Since the input consists of summary descriptions rather than the transcript, the input size is small and fits within the context window even for multi-hour lectures.

Continuing the same hypothetical example: the input to the GlobalPlanner is a list of several dozen strings of the form “Definition of the limit of a sequence and its informal interpretation”, “Arithmetic properties of limits”, “Examples of computing limits”, from which the agent might assemble a chapter “The limit of a sequence” with the corresponding subsections. The full lecture text is at no planning step presented to the model.

3.5. Global clustering

Each local cluster is assigned to one of the chapters of the plan. For matching, the multilingual model multilingual-e5-small vectorizes, on the one hand, the chapters of the plan (heading

together with description, with the `query:` prefix) and, on the other, the text of each local cluster itself (with the `passage:` prefix); a cluster is assigned to the chapter whose embedding is closest to it under cosine similarity.

On top of this pairwise choice, chronological smoothing is applied: since the chapters of the plan are ordered along the course of the lecture, transitions between adjacent clusters should be monotonic. A "jump backward" to an earlier chapter proposed by the model is ignored, while a single "jump forward" is accepted as a genuine transition only if confirmed by neighbouring clusters — otherwise it is treated as an embedder outlier and discarded. The union of all local clusters assigned to a single chapter forms a global cluster — a thematically grouped set of transcript fragments ready for synthesis.

3.6. Conspect synthesis

At this stage a MapReduce scheme is applied: the global cluster is split into chunks of fixed size with overlap (≈1433 tokens with an overlap of ≈512 tokens — 0.7 and 0.25 of the generation limit of 2048 tokens; length is measured by the model's own tokenizer), and the text is written chunk-by-chunk by the Synthesizer agent.

In each individual call the synthesizer sees only four things: the heading of the current chapter; the current transcript chunk; the "tail" of the previous chunk — the last 50 words for stitching the boundary; and the names of entities already introduced into the conspect.

```
seen_str = ", ".join(already_seen_themes) or "Nothing has been covered yet."
combined_context = (
    f"[ALREADY COVERED]: {seen_str}\n"          # registry of entity names
    f"[END OF PREVIOUS CHUNK]: {last_tail}"     # tail of the previous chunk
)
prompt = self._build_prompt(
    chunk=chunk,                          # current transcript chunk
    cluster_topik=topik,                  # heading of the current chapter
    previous_context=combined_context,
)  # simplified
```

The name registry is maintained by an auxiliary agent, the Extractor: after each written fragment it extracts the names of new entities from it and appends them to a global registry shared across the whole lecture. The synthesizer sees this registry and does not re-introduce already-explained concepts, which eliminates the duplication of definitions.

```
# after each written chunk
last_tail = " ".join(synthesize_chunk.split()[-50:])      # tail for stitching
extracted = self._extractor.run(synthesizer_chunk=synthesize_chunk)
for term in extracted.get("extracted_entities", []):
    already_seen_themes.add(term.lower())                 # append to the registry
```

Knowing its position in the document through the global plan and the state of the entity registry, the agent produces dry academic text without meta-narrative or artifacts of oral speech. The result of the block is a conspect in JSON format (key — chapter topic, value — its text), exported to Markdown.

Here it is appropriate to answer a natural objection: if the synthesizer already compresses the previous context down to a tail and a list of names, why not apply the same trick directly to the entire lecture, bypassing planning and clustering? Because the structure is carried not by this transfer but by the two inputs the synthesizer receives upstream in the pipeline. First, each call is written under the heading of a specific chapter from the global plan — without planning, the synthesizer has neither a heading nor a document skeleton under which to place the text. Second, the input is not a raw chronology but a transcript already cut along chapters: the stitching tail is reset at the start of each chapter, so it never glues together two different topics, whereas with a linear cut the chunk boundaries would fall in the middle of a topic. The name registry itself, meanwhile, is a deliberately minimal tool against the re-introduction of concepts: it is global over the whole lecture and carries only entity names without definitions (see Section 8), that is, it suppresses duplication but does not set structure. In other words, so meagre a transfer suffices precisely because the global structure is set by the plan and the chapter-aligned segmentation, not by the transfer itself.

3.7. Visualization generation

Visualization is moved into a separate final block and split into two roles.

The GraphPlanner agent analyzes the finished Markdown and places placeholders of the form `[GRAPH_TYPE: ...]` at semantically appropriate locations; the insertion points are determined by a normalized search over quotations, and the text of the conspect is not modified.

The Grapher agent, for each placeholder, produces an executable Python script that builds the image. The script is executed in an isolated subprocess with a stripped-down environment and a 15-second timeout. This is the only component of the system that uses self-correction: on an execution error the agent receives the stderr and the previous broken code and repeats generation, increasing the temperature by 0.1 at each attempt (starting from 0) — up to three attempts in total.

```
original_temp = self._gen_config.temperature  # 0
for attempt in range(self._app_config.re_try_count):            # up to 3 attempts
    self._gen_config.temperature = original_temp + attempt * 0.1  # 0 → 0.1 → 0.2
    code = self._generate_graph_code(description, target, error_message, bad_code)
    is_success, stderr_text = self._code_call(code, target, script_path)
    if is_success:
        break
    bad_code, error_message = code, f"Your code failed with an error:\n{stderr_text}"
```

Successfully generated images are saved as PNG. At the final stage the placeholders are replaced with references to the images, and the document is converted into a paginated PDF.

4. Hardware-oriented design

The total size of the weights of all pipeline models (STT, two embedding models, two LLMs) exceeds 8 GB, so simultaneous resident placement in video memory is impossible. A strict policy is applied: at any moment no more than one component resides in VRAM. The STT component finishes transcription and is fully unloaded before the LLM agents are initialized, the embedding model is unloaded after clustering, and so on.

Technically, this policy is enforced by isolating the stages across processes: the `run()` method of each stage executes in a separate child process that terminates at the end of the stage — whereupon the operating system releases all memory it occupied, including VRAM. This is exactly why the blocks exchange data through on-disk artifacts (Section 3.1): the result of each stage is persistent, and the next process need not (and cannot) keep the previous model in memory. Peak VRAM consumption is determined not by the sum of the models but by the most resource-intensive component. The price paid is the additional time for loading/unloading.

The target configuration is a GPU with 8 GB VRAM or more. On the test bench, an NVIDIA RTX 3050 (8 GB), the full path from an audio file to a finished PDF for a lecture lasting ≈1 h 30 min completes in approximately 35–40 minutes.

5. Conspecting paradigms

To formalize the notion of a "high-quality conspect", seven paradigms are formulated. The first four set structural-and-content requirements, the last three are criteria of practical quality. A limitation should be stated: these same paradigms serve both as design guidelines for the pipeline and as criteria for its evaluation, so high scores on them partly reflect the alignment of goal and measure rather than only an independent confirmation of quality; comparison against a reference evaluated by the same criteria partly mitigates this effect.

- **P1. Modular semantic architecture.** A conspect is not a linear retelling but a structured knowledge base with explicit functional blocks (definitions, theorems, proofs, corollaries, examples), allowing navigation without a full read-through.
- **P2. Objectivization and depersonalization.** The absence of meta-narrative (greetings, digressions, turns of phrase such as "next we will consider", "the lecturer moves on to") and of artifacts of oral speech; the exposition operates with facts and logic.
- **P3. Mathematical and terminological rigour.** Formulas and symbols are a full-fledged language; an approximate verbal description of formulas is unacceptable, and strict notation is required (LaTeX standards).
- **P4. Spatial-visual integration.** Where the material requires graphical representation, there must be designated places for integrating visual content with a description of what is depicted, rather than a verbal description of complex spatial objects.
- **P5. Factual accuracy.** Reproduction of facts, definitions, and statements without conjecture; substituting plausible but erroneous formulations is unacceptable — every statement must be contained in the original lecture.
- **P6. Informational completeness.** Reflection of the semantic content of the lecture, not just the correctness of what made it into the conspect; the severity of gaps is assessed.
- **P7. Self-sufficiency.** The finished conspect is comprehensible to a reader who did not attend the lecture and does not require recourse to the original recording.

6. Experimental methodology

Evaluation method. The LLM-as-a-judge methodology is used: the judge model Qwen3 Max Preview evaluated the conspect according to a specialized prompt, receiving as input the original transcript and the conspect under evaluation (PDF).

Reference. Gemini 3.1 Pro (the SOTA model at the time of this work), which processed the same transcript single-call under a detailed prompt. The proposed decompositional local scheme is thereby compared against direct feeding of the full context to a top cloud model.

Dataset. 10 lectures, 5 subject domains with 2 lectures each: algorithms, machine learning, calculus, biology, chemistry.

Judge prompt. The transcript is declared the sole source of truth: the conspect is evaluated strictly relative to what was said, without invoking external knowledge and without penalty for the absence of what the lecturer did not say; omitting organizational remarks, filler words, and repetitions is not counted as an error. For each paradigm a definition, signs of violation, and a graded 1–10 scale are specified. For P5 a structured list of errors is formed, divided into critical/significant/minor; for P6 the judge first compiles a list of the key semantic units of the transcript and labels each by coverage status (covered_deep / covered / partial / missed), and only then assigns a score. The answer is returned strictly in JSON with a justification and a quotation for each item, which ensures the reproducibility of the analysis.

7. Results

The average score across the seven paradigms was 6.87/10 for LCW against 8.84/10 for Gemini 3.1 Pro, which corresponds to ≈78% of the reference quality at zero inference cost and in fully offline mode on a GPU with 8 GB VRAM. The scores were obtained over a single run of the judge on a sample of 10 lectures, so the reported values are not accompanied by an estimate of variance and serve as a guide rather than a precise measurement (more in Section 8). The results by domain are given in Table 1.

Table 1. Scores across 10 lectures with per-domain subtotals and an overall average (LCW / Gemini 3.1 Pro / % of the reference).

#	Topic	Domain	LCW	Gemini 3.1 Pro	LCW %
1	Introduction. Basic Python constructs	Algorithms	8.14	8.57	95%
2	Boolean algebra. Branching	Algorithms	6.00	7.86	76%
	Algorithms		**7.07**	**8.22**	**86%**
3	Introduction. Basic concepts and notation	Machine learning	5.43	8.43	64%
4	Metric classification methods	Machine learning	6.57	8.29	79%
	Machine learning		**6.00**	**8.36**	**72%**
5	Basic definitions of calculus	Calculus	7.71	8.86	87%
6	A function and its graph. Sets of integers and rationals	Calculus	6.71	8.71	77%
	Calculus		**7.21**	**8.79**	**82%**
7	The definition of life. Micro- and macroelements	Biology	7.86	10.00	79%
8	Biological membranes. Secondary metabolites	Biology	7.00	9.57	73%
	Biology		**7.43**	**9.79**	**76%**
9	Basic concepts of chemistry	Chemistry	8.00	9.71	82%
10	The hydrogen atom and many-electron atoms	Chemistry	5.29	8.43	63%
	Chemistry		**6.65**	**9.07**	**73%**
	Average		**6.87**	**8.84**	**78%**

With two lectures per domain, the ordering of domains is illustrative and is not interpreted.

A breakdown across the seven paradigms is given in Table 2 (cell format — LCW / Gemini 3.1 Pro; lecture numbers correspond to Table 1).

Table 2. Scores across the seven paradigms P1–P7 for each of the 10 lectures, and the average (LCW / Gemini 3.1 Pro).

Lecture	P1	P2	P3	P4	P5	P6	P7	Average
1	9/9	10/10	8/9	3/4	9/10	9/9	9/9	8.14/8.57
2	8/9	8/10	3/8	3/3	4/10	8/8	8/7	6.00/7.86

Lecture	P1	P2	P3	P4	P5	P6	P7	Average
3	6/10	9/10	5/9	2/2	4/10	7/9	5/9	5.43/8.43
4	7/9	5/10	5/8	3/3	9/10	9/9	8/9	6.57/8.29
5	9/10	10/10	9/9	3/4	5/10	9/10	9/9	7.71/8.86
6	7/9	9/10	4/8	1/6	9/10	9/9	8/9	6.71/8.71
7	8/10	10/10	6/10	3/10	10/10	9/10	9/10	7.86/10.00
8	6/10	10/9	6/10	1/8	8/10	9/10	9/10	7.00/9.57
9	9/10	9/10	7/9	3/9	9/10	10/10	9/10	8.00/9.71
10	5/10	6/10	4/6	1/5	5/10	9/9	7/9	5.29/8.43
Average	**7.4/9.6**	**8.6/9.9**	**5.7/8.6**	**2.3/5.4**	**7.2/10.0**	**8.8/9.3**	**8.1/9.1**	**6.87/8.84**

Strengths. The closest match to the reference is on informational completeness (P6: 8.8 / 9.3) and self-sufficiency (P7: 8.1 / 9.1). This is consistent with the central hypothesis: two-level hierarchical planning effectively solves the coverage-completeness task ("not missing a topic") even in the absence of end-to-end context, and the resulting conspect reads without recourse to the original recording. It is significant that the characteristics most vulnerable to the long-context problem turned out to be the least affected. It should be borne in mind that high completeness is partly built into the method itself — the global plan by construction enumerates the topics — so the closeness to the reference on P6 and P7 is only partly an independent confirmation of quality rather than a purely unexpected effect.

Weaknesses. The largest shortfall relative to the reference is on factual accuracy (P5: 7.2 / 10.0) and mathematical rigour (P3: 5.7 / 8.6). Both characteristics are sensitive to model size: a quantized 8B model in GGUF format yields to large cloud models when working with complex mathematical notation and narrow terminology. This limitation lies at the model level and is not removed by the pipeline architecture.

Visualization. Paradigm P4 shows low values for both systems (LCW: 2.3; Gemini: 5.4): automatically choosing an appropriate chart type and generating it correctly is an objectively hard task regardless of model size. The near-half values for the SOTA reference indicate that the bottleneck here is the task itself, not only the size of the local model.

8. Limitations

Methodological. The applied LLM-as-a-judge methodology was not calibrated against human expert assessment, so the judge model could make errors. The judge was used through a web interface (not via an API) and was run once per lecture, which does not account for the variance of the scores. The sample size (10 lectures) is sufficient for a primary evaluation but insufficient for statistically significant conclusions. In addition, two known biases of the LLM-as-a-judge methodology were not controlled: the order effect (in what order the LCW and reference conspects were presented to the judge) and the influence of format and length (the judge evaluated PDFs, so the layout and document size could partly affect the assessment of content). For these reasons the quantitative results of Section 7 should be treated as indicative.

Comparison design. The reference (Gemini 3.1 Pro, single-call) and LCW differ along two axes — model size and architecture. Isolating the contribution of decomposition specifically by a "same model single-call versus the same model in LCW" comparison was not possible for an objective reason: a single-call of the same model is infeasible on the target hardware (Section 1). Therefore the quantitative quality reference is a cloud model, not a single-call; the obtained ≈78% characterizes closeness to a practically available alternative (the cloud) but does not prove a causal quality advantage of decomposition over a single-call, which on 8 GB is simply unavailable. Comparing LCW with other decompositions feasible on 8 GB (retrieval

approaches, sliding window) remains future work.

Architectural. The entity registry passes the synthesizer only the names of previously introduced entities, but not their definitions: the agent "knows" of the fact that a term was mentioned but does not have its original interpretation, which in some cases can give rise to inaccuracies in exposition. An effective compression of the previous context, more informative than a list of names, remains an open question. Furthermore, errors in global clustering can lead to a local cluster being assigned to the wrong chapter and a fragment landing in a semantically inappropriate section.

Engineering and domain. Clustering is performed over the raw transcription without prior markup, which may affect segmentation quality. The system is optimized exclusively for Russian-language lectures in the exact, natural, and technical sciences; the humanities and other languages fall outside the scope of the task addressed.

9. Conclusion

Established. On a consumer GPU with 8 GB VRAM, direct (single-call) processing of a long lecture is practically infeasible within the resource budget — memory and speed (Section 1) — and therefore decomposition, which keeps the context of each call small, acts as a necessary condition for operability rather than a quality optimization. The proposed LongConspectWriter pipeline realizes such a decomposition, runs on this hardware, and attains ≈78% of the quality of the cloud SOTA model Gemini 3.1 Pro at zero inference cost and with fully local processing (evaluated on 10 lectures). Beyond simple map-reduce, which yields only feasibility, two mechanisms are responsible for conspect quality: two-level planning of the structure before synthesis, and the registry of named entities.

Hypothesis. The preliminary data allow us to put forward — as a testable conjecture rather than a proven statement — a hypothesis about the separation of quality axes: structure-before-synthesis architectures recover predominantly the organizational component (coverage, structure, self-sufficiency) to a level close to a large model, whereas the gap concentrates in content-level accuracy (facts, formulas, terminology), which is limited by the model's capacity and is not closed by the architecture. Put more simply, by "quality axes" we mean two different properties of a conspect: how complete and well-organized it is, and how accurate it is in detail; decomposition, it appears, helps the former but not the latter. The sample size (10 lectures) is sufficient to demonstrate operability but insufficient to confirm this hypothesis as a generalization.